

Assignment 1

XINNI SONG U7439250

TALKING WITH U7439262 AND U7457841 AND SOME IDEAS

FROM CHATGPT

Assignment 1 of COMP4528 report

Task-1: Basic Image I/O (2 marks)

In this task, given the three images shown in Figure 1, you are asked to:

1)

The guideline is as follow:

- 1)Open image1, image2, image3 from the folder and store it in the variable img1, img2, img3
- 2)Resize img1 ,img2, img3 to 1024x720 pixels
- 3)Save img1 ,img2, img3 as a new file named image1.jpg

2)

a)

The guideline is as follow:

- 1)Open image1 from the folder and store it in the variable img1
- 2)Resize img1 to 384x256 pixels
- 3)Save img1 as a new file named image1.jpg

b)

Each RGB color image is built by three channels.

0 index channel is Red channel.

1 index channel is Green channel.

2 index channel is Blue channel.

So we can get the three grayscale image by traversing all pixels of the image and split every pixel in three channels. The pseudocode is as follow

Pseudocode

Get the width and height of the original image

width, height = img.size

Create new blank grayscale images for each color channel

r, g, b = Image.new('L', (width, height))

Split the RGB channels and put each channel in the corresponding grayscale image

for x in range(width):

for y in range(height):

pixel = img.getpixel((x, y))

r.putpixel((x, y), pixel[0])

g.putpixel((x, y), pixel[1])

b.putpixel((x, y), pixel[2])



Figure 1



Figure 2



Figure 1

Figure 1 is the gray image of R channel

Figure 2 is the gray image of G channel

Figure 3 is the gray image of B channel

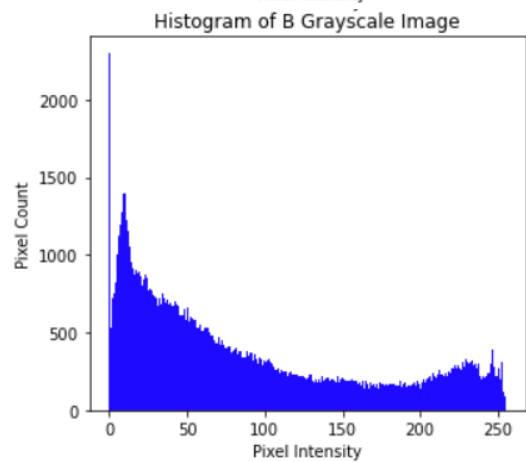
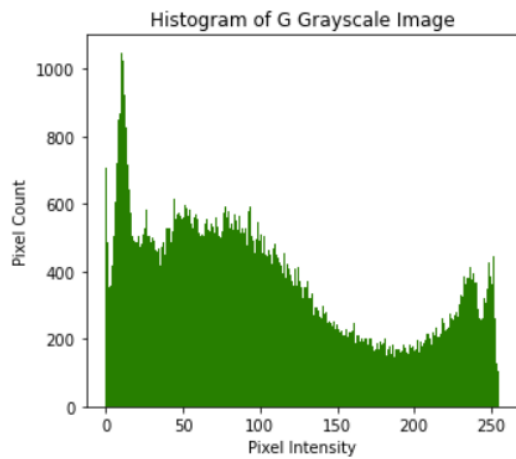
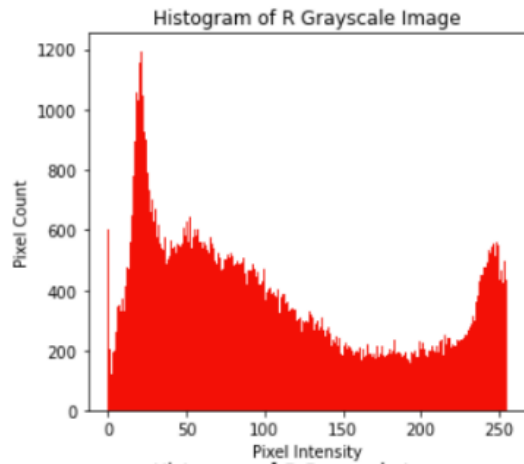
c) Compute the histograms for each of the grayscale images and display the 3 histograms (0.2 marks for each histogram, 0.6 marks in total).

For red, green, blue channels

```
hist, _ = np.histogram(np.array(r).flatten(), bins=256, range=(0, 255))
```

```
hist, _ = np.histogram(np.array(g).flatten(), bins=256, range=(0, 255))
```

```
hist, _ = np.histogram(np.array(b).flatten(), bins=256, range=(0, 255))
```



d) Apply histogram equalization to the individual grayscale channels of red, green, and blue, followed by merging the three channels to generate the coloured image. Display three new histograms after equalization and the new coloured image (0.15 marks for each histogram and image, 0.6 marks in total).

To get the equalization of three individual grayscale channels,

Step 1: Compute the PDF of the input image

```
histogram = compute_histogram(image)
```

```
pdf = histogram / total_pixels
```

Step 2: Compute the CDF of the PDF

```
cdf = compute_cdf(pdf)
```

Step 3: Compute the transformation function

```
L = 256 # total number of gray levels
```

```
mapping = round((L-1) * cdf)
```

Step 4: Apply the transformation to the input image

```
output_image = apply_transformation(image, mapping)
```

functions explain

```
def compute_histogram(image):
```

```
    # Compute the histogram of the image
```

```
    # ...
```

```
def compute_cdf(pdf):
```

```
    # Compute the cumulative sum of the PDF
```

```
    # Normalize the CDF so that it ranges from 0 to 1
```

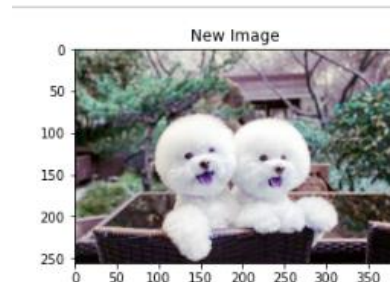
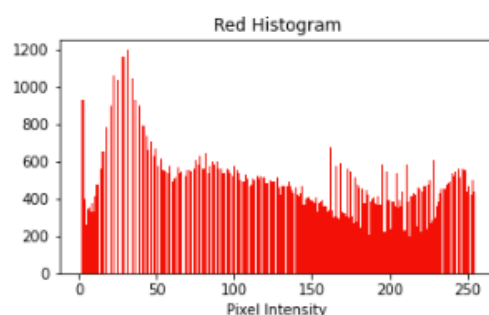
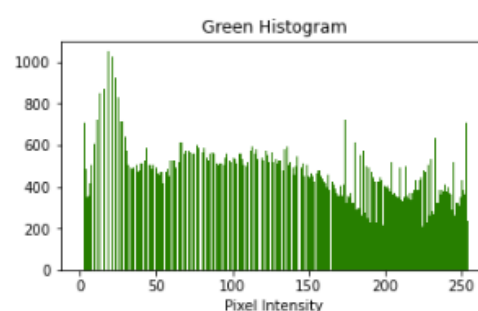
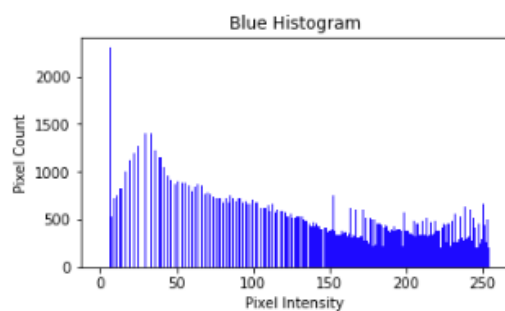
```
    # ...
```

```
def apply_transformation(image, mapping):
```

```
    # Replace each pixel in the input image with its mapped value
```

```
    # ...
```

Reference: P25 of image formation slide



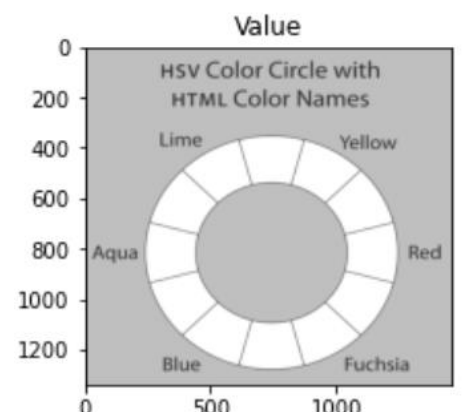
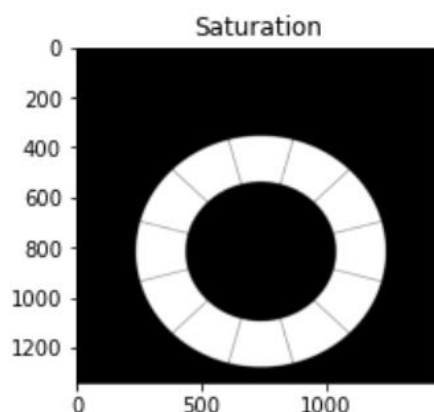
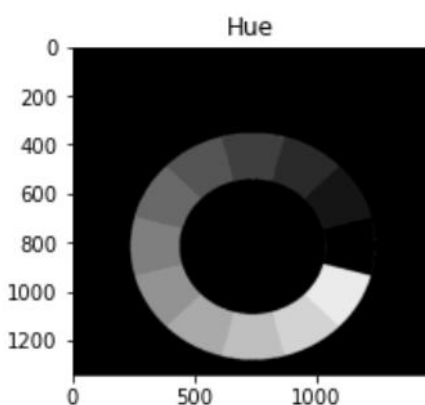
Task-2: Colour space conversion (5 marks)

1)

The pseudocode is as follow

```
function cvRGB2HSV(img):  
    # Normalize pixel values to the range [0, 1]  
    img = convert_to_float32(img) / 255.0  
    # get the red, green, and blue channels  
    R = img[:, :, 0]  
    G = img[:, :, 1]  
    B = img[:, :, 2]  
    # Compute the minimum and maximum values of the three channels  
    min_channel = np.minimum(np.minimum(R, G), B)  
    max_channel = np.maximum(np.maximum(R, G), B)  
    # Compute the V channel  
    V = max_channel  
    # Compute the S channel  
    S = np.zeros_like(V)  
    non_zero_mask = V != 0  
    C = max_channel - min_channel  
    S[non_zero_mask] = C[non_zero_mask] / V[non_zero_mask]  
    # Compute the H channel  
    H = np.zeros_like(V)  
    H[V == R] = (60 * ((G[V == R] - B[V == R]) / C[V == R]))  
    H[V == G] = (60 * ((B[V == G] - R[V == G]) / C[V == G])) + 120  
    H[V == B] = (60 * ((R[V == B] - G[V == B]) / C[V == B])) + 240  
    H[C == 0] = 0  
    # Convert hue to range [0, 1]  
    H /= 360.0  
    # Stack H, S, V to get the HSV image  
    hsv = np.dstack((H, S, V))  
    # Convert back to uint8  
    hsv = convert_to_uint8(hsv)  
    # Return the HSV image  
    return hsv
```

Use the `cvRGB2HSV` function above we get three graphs as follow:



2)

The way of distinguish the five region is k-means.

k-means clustering is a method of vector quantization, originally from signal processing, that aims to partition n observations into k clusters in which each observation belongs to the cluster with the nearest mean (cluster centers or cluster centroid), serving as a prototype of the cluster. (idea from <https://piazza.com/class/ldy9z5enl8736h/post/56>)

(From Wikipedia, link: https://en.wikipedia.org/wiki/K-means_clustering)

For this question, we get the H value of the Figure 2(b), and we see convert H as an array. Put the array into k-mean algorithm to cluster them as five different groups. And the result of the five groups is the five color regions in Figure 2(b).

The pseudocode is as follow

```
# Use rgb2hsv() function to obtain the HSV values of the image.
```

```
hsv = rgb2hsv(np.array(fig_b))
```

```
# Extract the H channel from the HSV values obtained in step 3 and 4.
```

```
H, _, _ = cv2.split(hsv)
```

```
# Reshape the H channel into a 1D vector and normalize it by dividing it by 255.
```

```
img_array = np.array(H)
```

```
img_vector = img_array.reshape(-1, 1)/255
```

```
# Use KMeans clustering algorithm to cluster the pixels in the 1D vector into 5 clusters.
```

```
kmeans = sklearn.cluster.KMeans(n_clusters=5, random_state=0).fit(img_vector)
```

```
# Use the labels divide them in five groups
```

```
Regions[] = divide_groups(img_array)
```

```
# Print the average H value of five regions in two different rgb2hsv function, the result is as follow.(Figure 2.2 1)
```

```
The average of the 1 region by my function is [0.05475149 0.05487375 0.05507124]
The average of the 2 region by my function is [0.54815235 0.54876816 0.5490049 ]
The average of the 3 region by my function is [0.25545681 0.2557295 0.25492077]
The average of the 4 region by my function is [0.14971527 0.14867389 0.14899287]
The average of the 5 region by my function is [0.00782433 0.00786195 0.00791837]
The average of the 1 region by built-in function is [0.05716615 0.05728845 0.05748328]
The average of the 2 region by built-in function is [0.54923988 0.54985337 0.5500872 ]
The average of the 3 region by built-in function is [0.25912648 0.25939559 0.25858603]
The average of the 4 region by built-in function is [0.15246487 0.15142608 0.15175051]
The average of the 5 region by built-in function is [0.00988824 0.00992909 0.00998651]
```

Figure 2.2 1

Task-3: Image Denoising via a Gaussian Filter (5 marks)

(1)

```
# Compute the the central square region
```

```
min_l = min(height, width)
```

```
x1 = int((width - min_l) / 2)
```

```
x2 = x1 + min_l
```

```
y1 = int((height - min_l) / 2)
```

```
y2 = y1 + min_l
```

```
# Crop the central square region(img4 is the original image4.jpg)
```

```
square_img = np.array(img4)[y1:y2, x1:x2]
```

Here are the results

Figure 3.1 1 is the original image4.jpg

Figure 3.1 2 is the center square image



Figure 3.1 1



Figure 3.1 2

(2)

```
# get the noise array
noise = normal_distribution(loc=0, scale=15, size=(512, 512))
# load the array form of image
img_array = get_array_from_image('image4.jpg')
# add the noise to image
noise_img = img_array + noise
# Scale the range
noise_img = clip_values(noise_img, min_value=0, max_value=255)
```

The noise result is as follow, Figure 3.2

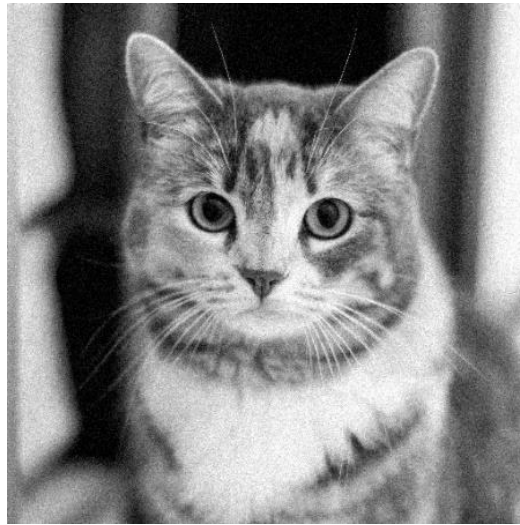


Figure 3.2

(3) The left image has no noise (Figure 3.3 1), and the right image adds some noise (Figure 3.3 2).

The no noise (Figure 3.3 1) image has many peaks in the image.

The noise (Figure 3.3 2) looks more smooth than the image with no noise.

The center of the noise (Figure 3.3 2) is high and shaped like a Gaussian distribution on a 2D map.

The reason is that Gaussian noise introduces random variations in the pixel values of an image.

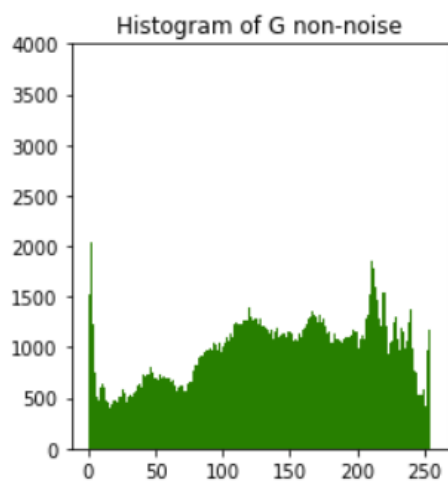


Figure 3.3 1

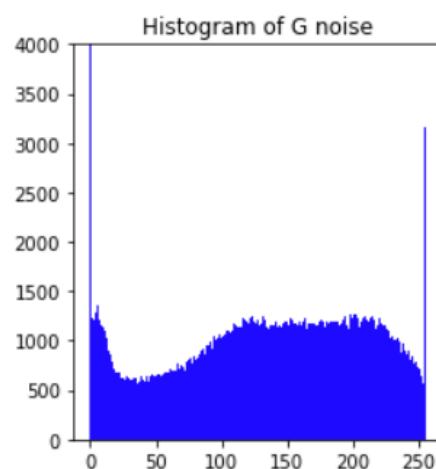


Figure 3.3 2

(4) my_Gauss_filter()

input: noisy_image, my 7x7 gauss_kernel

output: output_image

Here is the pseudocode of **my_Gauss_filter()**

```
function gaussian_filter(image, kernel):
```

```
    # Get the width and height of the image
```

```
    width, height = get_image_size(image)
```

```
    # Create an empty output image
```

```
    output_image = create_empty_image(width, height)
```

```
    # Pad the image with borders
```

```
    padded_image = pad_image(image, kernel_size)
```

```
    # Loop through each pixel in the image, use the kernel multiple the image pixels
```

```
    for x from 0 to width - 1:
```

```
        for y from 0 to height - 1:
```

```
            # Extract the region around the pixel, start from the first 7*7 pixel block in the upper left  
            corner of the padded image
```

```
            region = get_pixel_region(padded_image, x, y, kernel_size)
```

```
            # Compute the weighted sum of the region using the Gaussian kernel
```

```
            weighted_sum = calculate_weighted_sum(region, kernel)
```

```
            # Normalize the result and write it to the output image
```

```
            output_image[x, y] = normalize_result(weighted_sum)
```

```
    # Return the output image
```

```
    return output_image
```

(5) The input is the noise image, and after dealing with Gaussian, we get Figure 3.5 1

We can see that the picture is blurred and the noise is not obvious.

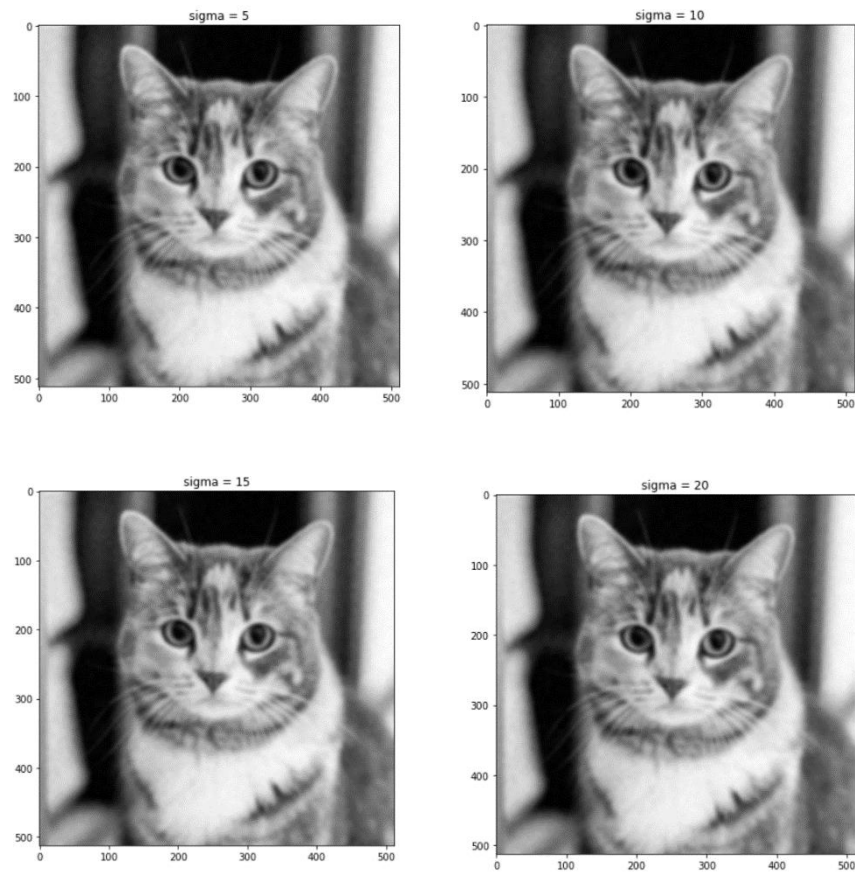


Figure 3.5 1

(6)

Here are the different results with different sigma of Gaussian filter.

When the sigma value is small, the size of the Gaussian kernel is small, and it will only smooth out the noise, while retaining the edges and details of the image; when the sigma value is large, the size of the Gaussian kernel is large, and it will smooth out the larger details and edges. , making the image more blurred



(7) The Figure 3.7 1 is the image from built-in function:

`cv2.GaussianBlur(img, (7,7), sigmaX=5, sigmaY=5, borderType=cv2.BORDER_REFLECT)`

The Figure 3.7 2 is the image by my gaussian filter.



Figure 3.7 1



Figure 3.7 2

Task-4: Implement your own 3x3 Sobel filter in MATLAB/Python (5 marks)

(1)

The explain of the Sobel filter is as follow.

```
function sobel_filter(gray_img, sobel_x, sobel_y)
# Get the height and width of the input image
height, width = gray_img.shape
# Create the x direction and y direction output images
output_image_x = new 2D array of size (height, width)
output_image_y = new 2D array of size (height, width)
# Get the size of the Sobel kernel
kernel_size = size of sobel_x
# Get the border size (half of the kernel size)
border_size = floor(kernel_size / 2)
# Pad the input image with mirror
padded_image = pad(gray_img, border_size, "mirror")
# Loop through each pixel in the input image
for I from 0 to height - 1:
for j from 0 to width - 1:
# Get the region of interest around the current pixel, start from the first sober size block in
the upper left corner of the padded image
roi = get_roi(padded_image, i, j, kernel_size)
# Compute the weighted sum of the ROI and the Sobel kernel ,store the gradient values in
the corresponding output image pixel
output_image_x[i, j] = sum(roi * sobel_x)
output_image_y[i, j] = sum(roi * sobel_y)
# Clip the output images to ensure pixel values are within the range [0, 255]
output_image_x = clip(output_image_x, 0, 255)
output_image_y = clip(output_image_y, 0, 255)
# Convert the data type of the output images to uint8, and return them
return to_uint8(output_image_x), to_uint8(output_image_y)
```

Figure 4.1 1 is the image2.jpg from built-in Sobel edge detection function

Figure 4.1 2 is the image4.jpg from built-in Sobel edge detection function

Figure 4.1 3 is the image2.jpg from my Sobel edge detection function `sobel_filter`

Figure 4.1 4 is the image4.jpg from my Sobel edge detection function `sobel_filter`

The result of my filter and its filter is similar.



Figure 4.1 1



Figure 4.1 2



Figure 4.1 1

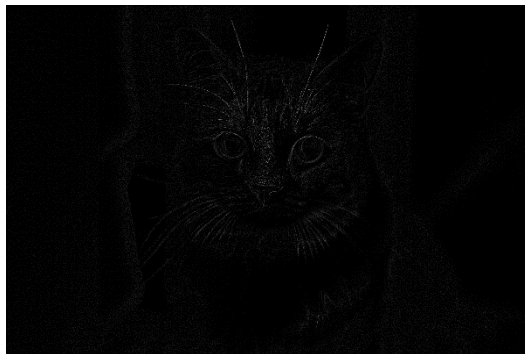


Figure 4.1 4

(2)

Figure 4.2 1 is the gradient orientation results of **image2** from mine and built-in sober function

Figure 4.2 2 is the gradient orientation results of **image4** from mine and built-in sober function

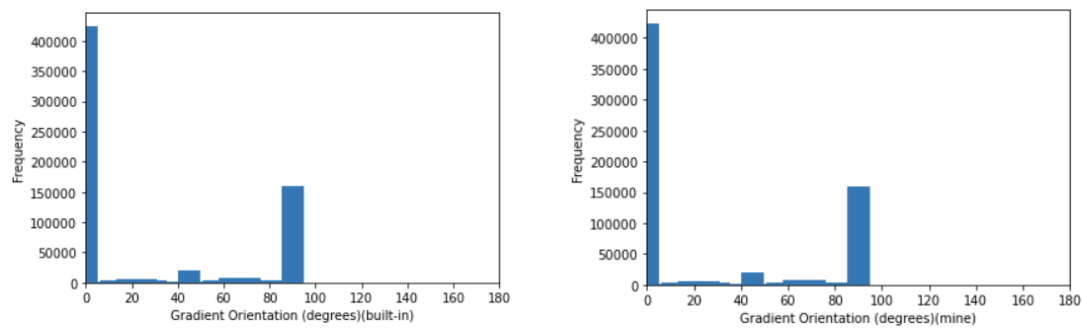


Figure4.2 2

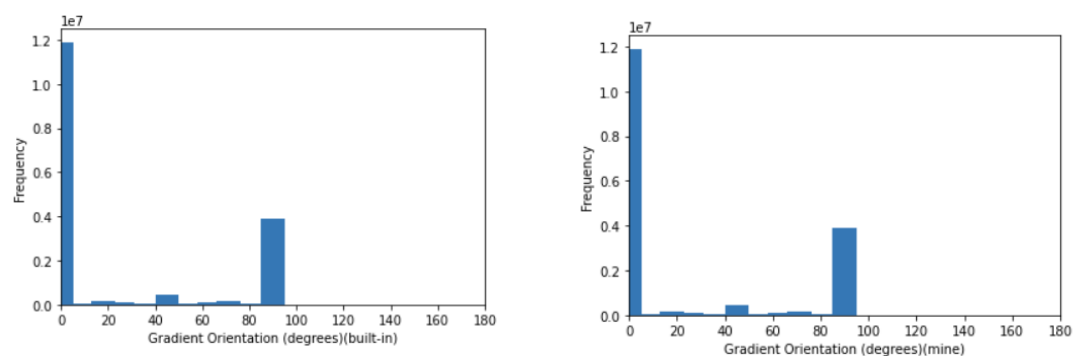


Figure4.2 1

The edges are mainly distributed between 0-10 degrees and 80-90 degrees. It can also be seen from the observation of the picture that there are many horizontal and vertical lines in the picture. The correct rate of edge detection may be affected by the noise in the picture, and it may also be affected by the sunlight in the photo.